19

Linear Algebra for AI

Historical Note

The linear algebra in a modern model is old. The dot product as a similarity score is Gibbs and Grassmann; the composition rule that makes a stack of layers collapse is Cayley's; the factorisation behind low-rank adaptation is Beltrami's; the graph Laplacian goes back to Kirchhoff's circuit laws of 1847 and its spectral theory to Fiedler in 1973. What is new is scale and hardware. Rosenblatt's perceptron of 1958 was a linear map with a threshold; the objection that stacking such things buys nothing, made forcefully by Minsky and Papert in 1969, is Proposition 19.3 below and is entirely correct. The resolution, which is to put a nonlinearity between the matrices, is not a deep piece of mathematics, and the field then waited thirty years for machines that could multiply large matrices quickly. This chapter is about where the linear algebra actually sits in the resulting systems, which is not always where the textbooks put it.

Every earlier chapter has ended with a remark about machine learning, and this one collects those remarks and pushes them further. The unifying observation is that essentially all the arithmetic in a large model is matrix multiplication, and essentially all the structure worth exploiting is one of three facts already proved: that composition is multiplication, that multiplication is associative, and that the best low-rank approximation is given by the singular values.

19.1 Data as a matrix

Represent each item, a word, an image patch, a user, by a vector in $\mathbb{R}^d$, and stack them as the columns of a matrix $X \in \mathbb{R}^{d \times N}$. Everything downstream is an operation on X.

Proposition 19.1 (Every linear score is a dot product)**.** *Let $f : \mathbb{R}^d \to \mathbb{R}$ be linear. Then there is exactly one $w \in \mathbb{R}^d$ with $f(x) = \langle x, w \rangle$ for all x, namely $w = \big(f(e_1), \dots, f(e_d)\big)$.*

Proof. By Theorem 7.5, f is determined by its values on the standard basis. With w as stated and $x = \sum_i x_i e_i$, linearity gives $f(x) = \sum_i x_i f(e_i) = \langle x, w \rangle$. Uniqueness holds because $\langle x, w \rangle = \langle x, w' \rangle$ for all x forces $w - w'$ to be orthogonal to everything, hence zero. □

Key Idea

That is the reason scores in these systems are dot products, and it is not a modelling choice. If the score of an item against a query is required to be linear in the query, then it *is* an inner product with some vector, and the only freedom is which vector. Similarity by cosine, from Definition 12.5, is the same quantity with the magnitudes divided out, which is what one wants when the length of an embedding carries frequency information rather than meaning.

Example 19.2 (Analogies as vector arithmetic)**.** *Example 2.4 noted that word embeddings famously satisfy relations such as king − man + woman ≈ queen. In the language of Chapter 2 the claim is that the displacement from* man *to* king *and the displacement from* woman *to* queen *are nearly the same vector, so the four points nearly form a parallelogram. Nothing about the training procedure asks for this, and how reliably it holds is disputed; what the linear algebra contributes is only the observation that if a relation is encoded by a fixed displacement, then it is recoverable by subtraction.*

19.2 Layers, and why a nonlinearity is needed

A fully connected layer is the affine map $x \mapsto Wx + b$. It is not linear, by Example 7.3, but appending a constant 1 to x and the column b to W makes it the linear map $\begin{pmatrix} W & b \end{pmatrix}$ acting on $\mathbb{R}^{d+1}$, which is why implementations carry a bias term as an extra input.

Proposition 19.3 (Depth without nonlinearity collapses)**.** *Let $T_i(x) = W_i x$ for $i = 1, \dots, L$. Then*

$$T_L \circ \cdots \circ T_1 \text{ is the single linear map } x \mapsto (W_L \cdots W_1)x,$$

and $\operatorname{rank}(W_L \cdots W_1) \le \min_i \operatorname{rank}(W_i)$.

Proof. The first statement is Theorem 7.19, that composition of linear maps is multiplication of their matrices. For the rank, $C(AB) \subseteq C(A)$ gives $\operatorname{rank}(AB) \le \operatorname{rank} A$, and transposing gives $\operatorname{rank}(AB) \le \operatorname{rank} B$; apply this along the product, as in Exercise 6.29. □

Key Idea

A stack of L linear layers has exactly the expressive power of one, no matter how large L is, and if any layer is narrow then the whole stack is capped at that rank. Interleaving a nonlinearity σ, applied entrywise, breaks the argument at its first step: $\sigma(W_2\sigma(W_1x))$ is not Mx for any M, because Theorem 7.19 needs both maps to be linear. So the activation function is not a detail of the architecture. It is the only reason depth exists.

Caution

It is worth being clear about what this chapter is not claiming. Linear algebra is the substrate of these models and not the model: everything that makes a network more than a matrix is in the nonlinearity, the loss, the data and the optimisation. What the linear algebra supplies is the arithmetic, the cost model and the structural constraints, which is a great deal but is not the same as an explanation of why the systems work.

19.3 Attention is three matrix products

Given n tokens, let $Q, K \in \mathbb{R}^{n\times d}$ hold the queries and keys as rows and let $V \in \mathbb{R}^{n\times d_v}$ hold the values. Attention computes

$$S = \frac{QK^t}{\sqrt{d}} \in \mathbb{R}^{n\times n}, \qquad P = \operatorname{softmax}(S) \text{ row by row}, \qquad \text{output} = PV \in \mathbb{R}^{n\times d_v},$$

where softmax exponentiates each entry and divides by its row sum.

Figure 19.1 asks which part of that work actually dominates. The projections cost $O(nd^2)$ and the attention matrix costs $O(n^2d)$, so the answer turns on how the sequence length compares with the width: at $d = 512$ the two are equal at $n = 1024$, and by $n = 4096$ the quadratic term is four times the other.

Figure 19.2 then carries the shapes through and shows where that quadratic term lives. Forming S costs about n^2d operations and multiplying by V costs about n^2d_v, both quadratic in the number of tokens, while everything else in a layer is linear in it.

Key Idea

Each entry $S_{ij} = \langle q_i, k_j \rangle / \sqrt{d}$ is a dot product, which Proposition 19.1 says is the general form of a linear score. The division by $\sqrt{d}$ is variance control: if the

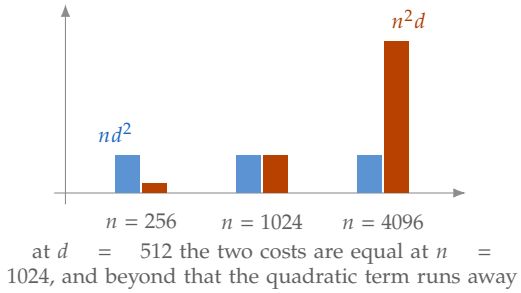


Figure 19.1. Where the cost of attention actually goes. Forming the queries, keys, values and output projection costs $O(nd^2)$, and forming the $n \times n$ attention matrix and applying it costs $O(n^2d)$. The first is linear in the sequence length and the second quadratic, so which dominates depends entirely on how n compares with d. At $d = 512$ they are equal at $n = 1024$; at $n = 4096$ the quadratic term is four times the other. This is the whole motivation for the approximations of Chapter 24: a low-rank or sparse stand-in for the attention matrix removes the n^2 and leaves a cost linear in the sequence length, which is why long-context models cannot simply use the exact computation.

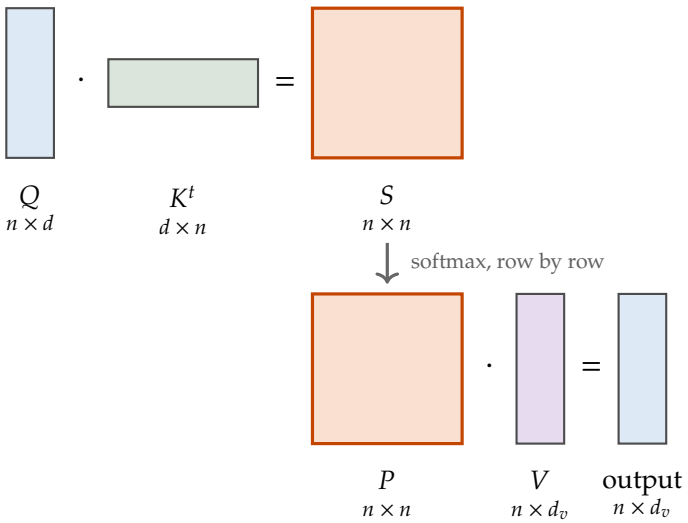


Figure 19.2. The shapes of attention. Forming S costs about n^2d operations and multiplying by V costs about n^2d_v; both are quadratic in the number of tokens n, while everything else in a layer is linear in it. Softmax acts within each row and changes no shape, but it is the step that prevents the two products from being regrouped, as the Insight below explains. Those square blocks are the reason long context is expensive.

entries of q and k are independent with unit variance then $\langle q, k \rangle$ is a sum of d such products and has variance d, so without the division the scores would grow with the width and drive softmax towards a hard maximum. And each row of P is nonnegative and sums to 1, so every output row is a weighted average of the rows of V: the same convex-combination structure as the Markov matrices of Chapter 10.

Insight

Here is associativity earning its keep. If the softmax were absent, the output would be $(QK^t)V$, and by Theorem 4.6 that equals $Q(K^tV)$. The left grouping costs about n^2d and forms an $n \times n$ matrix; the right grouping forms only a $d \times d_v$ matrix and costs about ndd_v, which is *linear* in the sequence length. For $n = 10^5$ and $d = 64$ that is the difference between 10^{11} operations and 10^8.
Every "linear attention" method is an attempt to collect that prize. The obstruction is precisely that softmax is not a linear map, so it cannot be moved across the regrouping; the methods replace it by something of the form $\varphi(q)^t\varphi(k)$, which can. Whether the replacement costs too much in quality is an empirical question, but the reason anyone tries is a theorem from Chapter 4.

19.4 Backpropagation is the adjoint

Let a network be a composition $f = f_L \circ \cdots \circ f_1$ followed by a scalar loss ℓ. Write J_i for the Jacobian of f_i at the point it is evaluated, so that by the chain rule the Jacobian of the whole is the product $J = J_L \cdots J_1$: the chain rule *is* matrix multiplication.

Proposition 19.4 (The gradient is the adjoint applied backwards)**.** *Let $y = f(x)$ and $\ell = g(y)$ with g scalar. Then*

$$\nabla_x \ell \;=\; J^t\,\nabla_y g \;=\; J_1^t J_2^t \cdots J_L^t\,\nabla_y g.$$

Proof. The chain rule says the derivative of ℓ at x is the linear functional $v \mapsto \langle \nabla_y g, Jv \rangle$. By Proposition 14.1 that equals $\langle J^t\nabla_y g, v \rangle$, and by Proposition 19.1 the vector representing a linear functional is unique, so $\nabla_x \ell = J^t\nabla_y g$. Transposing the product reverses the order, by Proposition 4.9. □

Key Idea

That reversal is the whole of backpropagation. The forward pass applies J_1, then J_2, and so on; the backward pass applies J_L^t, then J_{L-1}^t, and so on. It runs backwards because the transpose of a product reverses it, which is Chapter 4, and it is a transpose in the first place because the gradient of a scalar is the adjoint applied to 1, which is Chapter 14. In PDE-constrained optimisation the identical algorithm is called the adjoint method, and it was in use there before anyone in machine learning had a name for it.

Figure 19.3 draws the two passes on one diagram, the forward one along the top and the backward one along the bottom, with each backward arrow the transpose of the forward derivative above it.

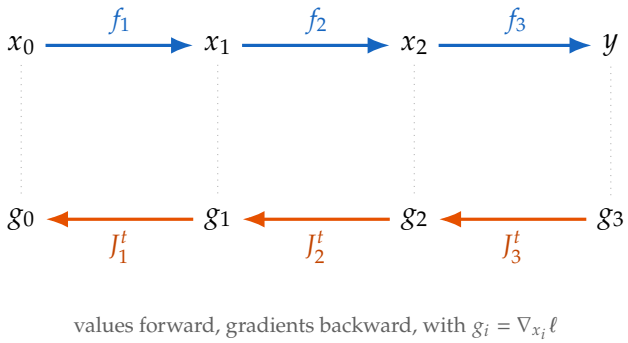


Figure 19.3. Proposition 19.4. The upper row is the forward pass and the lower row the backward pass, and each backward arrow is the transpose of the corresponding forward derivative. For a scalar loss this one backward sweep produces every partial derivative at a cost comparable to a single forward pass, whereas computing the same derivatives by propagating forwards would need one sweep per input variable. That asymmetry, and not any property of neural networks, is why reverse mode is universal.

Proposition 19.5 (Weight gradients are outer products). *For a layer $y = Wx$ and a scalar loss ℓ,*

$$\nabla_W \ell \;=\; (\nabla_y \ell)\, x^t,$$

a matrix of rank one. Summed over a batch of B examples, the gradient has rank at most B.

Proof. $y_i = \sum_j W_{ij} x_j$, so $\partial \ell / \partial W_{ij} = (\partial \ell / \partial y_i)\, x_j$, which is the (i, j) entry of the outer product. Rank one is Exercise 16.21. For a batch, the column space of the sum is contained

in the sum of the individual column spaces, which is a subspace by Proposition 5.10 and is spanned by B vectors, so its dimension is at most B by Theorem 6.14(b). □

19.5 Low rank

Example 19.6 (Low-rank adaptation). *To adapt a pretrained weight matrix $W \in \mathbb{R}^{d\times d}$ without retraining it, replace it by $W + BA$ with $B \in \mathbb{R}^{d\times r}$ and $A \in \mathbb{R}^{r\times d}$ for some small r. The number of trained parameters falls from d^2 to $2dr$: at $d = 4096$ and $r = 8$ that is* 16.8 *million against* 65 536*, a factor of* 256*.*
What linear algebra contributes is the bound on what this can cost. If ΔW is the update one would ideally make, then by Theorem 16.12 the best rank-r approximation to it is in error by exactly $\sigma_{r+1}(\Delta W)$ in the spectral norm, so the method loses little precisely when the singular values of the ideal update decay quickly.

Caution

That is a conditional statement and it is worth not overselling. Proposition 19.5 says each gradient step contributes a matrix of rank at most the batch size, which is suggestive, but a sum of many such steps can perfectly well have full rank, so the proposition does not prove that the accumulated update is low rank. Eckart–Young quantifies the loss *if* the singular values decay; whether they do is an empirical fact about particular models and tasks, not a theorem. The honest summary is that low-rank adaptation is a bet whose payoff linear algebra can measure but not guarantee.

19.6 Graphs and their spectra

Let G be an undirected graph on n vertices with adjacency matrix A and degree matrix $D = \operatorname{diag}(d_1, \dots, d_n)$. The *Laplacian* is $L = D - A$.

Proposition 19.7 (The quadratic form of the Laplacian). *L is symmetric, and for every $x \in \mathbb{R}^n$,*

$$\langle Lx, x\rangle \;=\; \sum_{\{i,j\}\in E} (x_i - x_j)^2 \;\geq\; 0,$$

so L is positive semidefinite. Moreover $Lu = \mathbf{0}$, where $u = (1, 1, \dots, 1)$ denotes the all-ones vector throughout this section.

Proof. Symmetry is clear. Expanding, $\langle Lx, x\rangle = \sum_i d_i x_i^2 - \sum_{i,j} a_{ij} x_i x_j$. Each edge $\{i, j\}$ contributes $x_i^2 + x_j^2$ to the first sum, since it raises both degrees by one, and $2x_i x_j$ to the second, since $a_{ij} = a_{ji} = 1$. Their difference is $(x_i - x_j)^2$. For the last claim, row i of L sums to $d_i - d_i = 0$. □

Theorem 19.8 (Zero eigenvalues count components)**.** *The multiplicity of the eigenvalue 0 of L equals the number of connected components of G, and $N(L)$ is spanned by the indicator vectors of those components.*

Proof. By Proposition 19.7, $Lx = \mathbf{0}$ implies $\langle Lx, x\rangle = 0$, hence $x_i = x_j$ for every edge, hence x is constant on each connected component. Conversely any such x has $\langle Lx, x\rangle = 0$, and since L is positive semidefinite this forces $Lx = \mathbf{0}$: for if $L = QDQ^t$ by Theorem 14.7, then $\langle Lx, x\rangle = \sum_i \lambda_i y_i^2$ with $y = Q^t x$, and with all $\lambda_i \geq 0$ the sum vanishes only when $\lambda_i y_i = 0$ for every i, that is when $Lx = \mathbf{0}$. So $N(L)$ consists exactly of the functions constant on components, which is the span of the component indicators, and those are independent because the components are disjoint. □

Proposition 19.9 (The Fiedler value)**.** *Let G be connected, so that $\lambda_1 = 0$ is simple with eigenvector u. Then the second smallest eigenvalue satisfies*

$$\lambda_2 \;=\; \min\left\{ \frac{\langle Lx, x\rangle}{\langle x, x\rangle} \;:\; x \neq \mathbf{0},\; x \perp u \right\},$$

attained at an eigenvector for λ_2, called the Fiedler vector.

Proof. Take an orthonormal eigenbasis $v_1 = u/\sqrt{n}, v_2, \ldots, v_n$ from Theorem 14.7 and write $x = \sum_i c_i v_i$. Orthogonality to u gives $c_1 = 0$, so as in the proof of Theorem 14.15,

$$\frac{\langle Lx, x\rangle}{\langle x, x\rangle} = \frac{\sum_{i \geq 2} \lambda_i c_i^2}{\sum_{i \geq 2} c_i^2} \;\geq\; \lambda_2,$$

with equality at $x = v_2$. □

Key Idea

Combine the two displays. Minimising $\langle Lx, x\rangle$ means minimising $\sum_{\text{edges}} (x_i - x_j)^2$, that is, asking adjacent vertices to receive similar numbers; the constraint $x \perp u$ forbids the trivial answer of giving everyone the same number. So the Fiedler vector is the most nearly constant nonconstant labelling of the graph, and splitting

the vertices by its sign cuts the graph where it is already nearly disconnected. That is spectral clustering, and it is Theorem 14.15 with one constraint added.

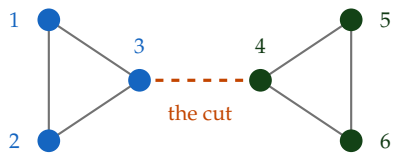
Example 19.10 (A path, exactly). *For the path on four vertices, with edges* 1–2, 2–3, 3–4,

$$L = \begin{pmatrix} 1 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & 1 \end{pmatrix},$$

whose eigenvalues are $2 - 2\cos(k\pi/4)$ *for* $k = 0, 1, 2, 3$, *namely* 0, $2 - \sqrt{2} \approx 0.586$, 2 *and* $2 + \sqrt{2}$. *The graph is connected, so* 0 *is simple, as Theorem 19.8 requires. The Fiedler vector has entries* $\cos\big((i - \tfrac{1}{2})\pi/4\big)$, *that is*

$$v_2 \approx (0.924,\ 0.383,\ -0.383,\ -0.924),$$

which the reader should check satisfies $Lv_2 = (2 - \sqrt{2})v_2$. *Its signs split* $\{1, 2\}$ *from* $\{3, 4\}$, *cutting the middle edge, which is the only sensible cut of a path.*



$\lambda_2 = \frac{1}{2}(5 - \sqrt{17}) \approx 0.44$;
the Fiedler vector is positive on the blue vertices and negative on the green

Figure 19.4. Spectral clustering on six vertices. The graph is two triangles joined by one edge, and the Fiedler vector, which by symmetry has the form $(a, a, b, -b, -a, -a)$, is positive on one triangle and negative on the other. Cutting where it changes sign removes the single bridging edge. The value $\lambda_2 \approx 0.44$ measures how nearly disconnected the graph is: adding more bridges would raise it, and removing the one would send it to zero, as Theorem 19.8 requires.

Insight

The same construction appears wherever data carries a graph. Spectral clustering partitions by the sign of the Fiedler vector or by several eigenvectors at once.

Graph neural networks propagate information by multiplying repeatedly by a normalised adjacency matrix, so the smoothing they perform is the power iteration of Section 9.6 and the well-known tendency of deep such networks to make all vertices alike is convergence to the dominant eigenvector. PageRank, mentioned in Chapter 9, is the stationary distribution of a random walk on a graph. In every case what is being computed is an eigenvector of a matrix built from the connectivity, and what is being exploited is that Chapter 14 guarantees a real orthonormal eigenbasis whenever that matrix is symmetric.

Open Problem 19.11 (Which solution does gradient descent choose?)**.** *Train a model with more parameters than data. Infinitely many parameter settings fit the data exactly, and they generalise very differently. Gradient descent, started from small weights, reliably picks a good one.* Which one, and why?

What is known. *In the purely linear case the answer is clean and provable: gradient descent on a least squares problem, started at zero, converges to the minimum ℓ_2 norm solution, because every update lies in the row space of A and so the iterate never leaves it. That is the projection theory of Chapter 13, and it is the entire content of "implicit regularisation" in one layer. Add a second layer, even with no nonlinearity, and the answer changes: gradient descent on a product of two matrices is biased towards the minimum* nuclear *norm solution, which is Chapter 35's convex surrogate for rank [284]. Beyond a few such cases, and in particular for any network with nonlinearities trained past the small-initialisation regime,* no characterisation is known.

Where the difficulty sits. *The bias is a property of the whole trajectory, not of the objective, so it cannot be read off from the loss surface. It depends on the initialisation scale, the step size, and the parameterisation, and changing any of them changes the answer. There is no known variational principle whose solution the trajectory finds, except in the special cases above.*

Where a reader might start. *Fit an underdetermined linear system by gradient descent from zero and check numerically that you land on the pseudoinverse solution, then repeat with a factored parameterisation $W = UV$ and watch the answer change to something of lower rank.* Two experiments, twenty lines of code, and the phenomenon is in front of you. *Chapter 61 is where the nonconvex geometry begins.*

Exercises

Exercise 19.12. *Let W_1 be 4×10, W_2 be 2×4 and W_3 be 10×2. What is the largest possible rank of $W_3 W_2 W_1$, and which layer is responsible? Give an example achieving it.*

Exercise 19.13. *Two embeddings satisfy $\|u\| = 3$, $\|v\| = 5$ and $\langle u, v \rangle = 12$. Find the cosine similarity and the angle. Now replace v by $10v$: which of the two quantities changes, and what does that say about when cosine is the right measure?*

Exercise 19.14. *For $n = 4096$, $d = 64$ and $d_v = 64$, count the multiplications in QK^t and in PV. Then count them for $Q(K^t V)$, ignoring softmax. At what value of n do the two totals cross?*

Exercise 19.15. *Show that softmax is not a linear map, by exhibiting vectors u, v with $\mathrm{softmax}(u + v) \neq \mathrm{softmax}(u) + \mathrm{softmax}(v)$. Deduce that the regrouping in the Insight of Section 19.3 is not available when softmax is present.*

Exercise 19.16. *For the layer $f(x) = Wx$ compute the Jacobian, and for $f(x) = \sigma(Wx)$ with σ applied entrywise show that the Jacobian is $\mathrm{diag}(\sigma'(Wx))\, W$. Write down the corresponding backward step.*

Exercise 19.17. *Verify Proposition 19.5 by hand for W of size 2×2, $x = (1, 2)$ and $\nabla_y \ell = (3, -1)$. Confirm that the resulting matrix has rank one and identify its column space.*

Exercise 19.18. *A weight matrix is 2048×2048. Compare the parameter counts of a full update and of a rank-16 update. If the singular values of the ideal update are $\sigma_i = 2^{-i/4}$, what fraction of $\|\Delta W\|_F$ does a rank-16 approximation retain?*

Exercise 19.19. *Write down the Laplacian of the triangle on three vertices and of the star on four vertices, and find all eigenvalues of each. Check Theorem 19.8 in both cases, and compare the two values of λ_2: which graph is better connected?*

Exercise 19.20. *Verify the claim in Figure 19.4 by setting up the two distinct eigenequations for a vector of the form $(a, a, b, -b, -a, -a)$, deriving $\lambda^2 - 5\lambda + 2 = 0$, and solving. Which root is λ_2, and what does the other root correspond to?*

References and further reading

This chapter is deliberately thin on machine learning and thick on which theorem is doing the work, so the reading naturally splits. For the models themselves the primary literature moves too fast for a textbook to track, and the reader is better served by the original papers on attention and on low-rank adaptation than by any secondary account. For the linear algebra, everything used here has already been proved in this book, and the references of the relevant chapters apply. Two additions are worth naming. Spectral graph theory has a literature of its own beginning with Fiedler's 1973 paper on algebraic connectivity, and Chung's monograph is the standard entry point. Automatic differentiation, of which backpropagation is the reverse mode specialised to a scalar output, is a subject with its own careful theory; Griewank and Walther is the reference, and reading it makes clear how little of the algorithm is specific to neural networks. Golub and Van Loan [263] remains the place to look for anything about the matrix computations underneath.